Home Church International - complete source code

Flask + SQLite. 12 automated tests passed on this exact code. Admin can upload, replace and delete images in every section (slides, Daily Hope, events, sermons, leaders, projects). Files are listed in this order: README.md, requirements.txt, requirements-dev.txt, .gitignore, .env.example, Dockerfile, docker-compose.yml, render.yaml, .github/workflows/ci.yml, app.py, templates/admin.html, templates/admin_ent.html, templates/admin_live.html, templates/admin_messages.html, templates/admin_nav.html, templates/admin_requests.html, templates/auth.html, templates/base.html, templates/give.html, templates/home.html, templates/list.html, templates/live.html, templates/member.html, templates/request.html, templates/sermon.html, static/css/site.css, static/js/site.js, tests/test_app.py.

Important: copying code out of a PDF can lose Python indentation. For GitHub, upload the project folder from homechurch.zip (it also holds the 3 church photos in static/img). Use this PDF as the readable reference and backup. Lines ending with a backslash were wrapped for printing only.

README.md

```
# Home Church International (Flask + SQLite)
Run locally: `pip install -r requirements.txt -r requirements-dev.txt`, then `pytest -q`, then
`ADMIN_EMAIL=you@x.org ADMIN_PASSWORD=your-long-password python app.py` and open http://localhost:5000 (admi \
    n: /admin).

## Put it on GitHub
1. Create an empty repository on github.com. 2. In this folder: `git init && git add . && git commit -m "Hom \
    e Church site" && git branch -M main && git remote add origin https://github.com/YOU/REPO.git && git pus \
    h -u origin main`.
3. GitHub Actions runs the tests automatically (.github/workflows/ci.yml).
Note: GitHub stores the code; GitHub Pages only serves static files and cannot run this Python app.

## Host it (connect the GitHub repo)
Render: New > Blueprint > pick the repo (uses render.yaml); set ADMIN_EMAIL, ADMIN_PASSWORD, GIVING_INFO. Up \
    loads and the database live under DB_PATH,
so they need a persistent disk (paid on Render) or another host with a volume (Railway, Fly.io, a VPS with ` \
    docker compose up -d --build`).
Back up the data folder (church.db + uploads/) regularly. Never commit .env.
```

requirements.txt

```
flask>=3.0
gunicorn>=22
Pillow>=10
```

requirements-dev.txt

```
pytest>=8
```

.gitignore

```
.env
data/
__pycache__/
*.pyc
.pytest_cache/
```

.env.example

```
SECRET_KEY=REPLACE_WITH_LONG_RANDOM
ADMIN_EMAIL=you@your-domain.org
ADMIN_PASSWORD=REPLACE_WITH_LONG_PASSWORD
COOKIE_SECURE=1
GIVING_INFO=Verified M-Pesa / bank details go here
```

Dockerfile

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt && useradd -r -u 10001 app && mkdir data && chown app dat \
    a
COPY . .
USER 10001
CMD ["gunicorn","-b","0.0.0.0:8000","-w","3","app:app"]
```

docker-compose.yml

```
services:
  app:
    build: .
    restart: unless-stopped
    env_file: .env
    volumes: [church_data:/app/data]
    ports: ["8000:8000"]   # put an HTTPS reverse proxy (Caddy/Nginx) in front for production
volumes:
  church_data:
```

render.yaml

```
services:
```

```
  - type: web
    name: home-church
    runtime: python
    buildCommand: pip install -r requirements.txt
    startCommand: gunicorn app:app
    disk: { name: church-data, mountPath: /var/data, sizeGB: 1 }   # persistent disk needs a paid Render pla \
        n
    envVars:
      - { key: DB_PATH, value: /var/data/church.db }
      - { key: COOKIE_SECURE, value: "1" }
      - { key: SECRET_KEY, generateValue: true }
      - { key: ADMIN_EMAIL, sync: false }
      - { key: ADMIN_PASSWORD, sync: false }
      - { key: GIVING_INFO, sync: false }
```

.github/workflows/ci.yml

```
name: tests
on: [push, pull_request]
jobs:
  test:
    runs-on: ubuntu-latest
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: "3.12" }
      - run: pip install -r requirements.txt -r requirements-dev.txt && pytest -q
```

app.py

```
import os, sqlite3, secrets, datetime as dt
from functools import wraps
from flask import Flask, send_from_directory, g, request, session, redirect, render_template, abort, flash,  \
    url_for
from werkzeug.security import generate_password_hash, check_password_hash
BASE = os.path.dirname(os.path.abspath(__file__))
app = Flask(__name__)
app.secret_key = os.environ.get("SECRET_KEY") or secrets.token_hex(32)
app.config.update(SESSION_COOKIE_HTTPONLY=True, SESSION_COOKIE_SAMESITE="Lax",
    SESSION_COOKIE_SECURE=os.environ.get("COOKIE_SECURE", "0") == "1", MAX_CONTENT_LENGTH=8*1024*1024)
DB = os.environ.get("DB_PATH", os.path.join(BASE, "data", "church.db"))
# entity -> (label, fields). Every admin-managed content type uses this one definition.
ENT = {
 "slides": ("Homepage slides", ["title","body","image_url","action_label","action_url","sort_order","starts_ \
     at","ends_at"]),
 "hope": ("Daily Hope", ["heading","scripture_reference","verse_text","reflection","prayer_prompt","publish_ \
     date","image_url"]),
 "events": ("Events", ["title","description","starts_at","location","image_url"]),
 "sermons": ("Sermons", ["title","preacher","scripture","recorded_at","video_url","summary","image_url"]),
 "leaders": ("Leaders", ["name","role_title","bio","image_url","sort_order"]),
 "projects": ("Projects", ["title","description","goal_text","image_url"]),
}
LONG = {"body","description","verse_text","reflection","prayer_prompt","summary","bio","goal_text"}
def db():
    if "db" not in g:
        g.db = sqlite3.connect(DB); g.db.row_factory = sqlite3.Row
        g.db.execute("PRAGMA foreign_keys=ON")
    return g.db
@app.teardown_appcontext
def close(_):
    d = g.pop("db", None)
    if d: d.close()
def init():
    os.makedirs(os.path.dirname(DB), exist_ok=True)
    c = sqlite3.connect(DB)
    for k, (_, f) in ENT.items():
        c.execute(f"CREATE TABLE IF NOT EXISTS {k}(id INTEGER PRIMARY KEY, "+",".join(f"{x} TEXT NOT NULL DE \
            FAULT ''" for x in f)+", published INTEGER NOT NULL DEFAULT 0)")
    for k, (_, f) in ENT.items():
        have = {r[1] for r in c.execute(f"PRAGMA table_info({k})")}
        for x in f:
            if x not in have: c.execute(f"ALTER TABLE {k} ADD COLUMN {x} TEXT NOT NULL DEFAULT ''")
    c.executescript("""
    CREATE TABLE IF NOT EXISTS users(id INTEGER PRIMARY KEY, email TEXT UNIQUE NOT NULL, password_hash TEXT  \
        NOT NULL,
      display_name TEXT NOT NULL, role TEXT NOT NULL DEFAULT 'MEMBER', enabled INTEGER NOT NULL DEFAULT 1, c \
          reated_at TEXT DEFAULT CURRENT_TIMESTAMP);
    CREATE TABLE IF NOT EXISTS live_service(id INTEGER PRIMARY KEY CHECK(id=1), title TEXT, stream_url TEXT, \
         status TEXT DEFAULT 'SCHEDULED', scheduled_at TEXT);
    INSERT OR IGNORE INTO live_service(id,title,status) VALUES(1,'Sunday Worship Service','SCHEDULED');
    CREATE TABLE IF NOT EXISTS requests(id INTEGER PRIMARY KEY, kind TEXT, name TEXT, contact TEXT, body TEX \
        T, consent INTEGER, status TEXT DEFAULT 'NEW', created_at TEXT DEFAULT CURRENT_TIMESTAMP);
    CREATE TABLE IF NOT EXISTS messages(id INTEGER PRIMARY KEY, user_id INTEGER, subject TEXT, body TEXT, cr \
        eated_at TEXT DEFAULT CURRENT_TIMESTAMP);
    CREATE TABLE IF NOT EXISTS comments(id INTEGER PRIMARY KEY, sermon_id INTEGER, user_id INTEGER, text TEX \
        T, rating INTEGER, created_at TEXT DEFAULT CURRENT_TIMESTAMP);
```

```
    CREATE TABLE IF NOT EXISTS audit(id INTEGER PRIMARY KEY, user_id INTEGER, action TEXT, entity TEXT, crea \
        ted_at TEXT DEFAULT CURRENT_TIMESTAMP);
    """)
    if not c.execute("SELECT 1 FROM users WHERE role='ADMIN'").fetchone():
        em = os.environ.get("ADMIN_EMAIL", "admin@localhost")
        pw = os.environ.get("ADMIN_PASSWORD") or secrets.token_urlsafe(12)
        c.execute("INSERT INTO users(email,password_hash,display_name,role) VALUES(?,?,?,'ADMIN')", (em, gen \
            erate_password_hash(pw), "Administrator"))
        if not os.environ.get("ADMIN_PASSWORD"): print(f"\n*** First admin created: {em} / {pw}  (shown once \
             - change it) ***\n")
    c.commit(); c.close()
def me():
    return db().execute("SELECT * FROM users WHERE id=? AND enabled=1", (session.get("uid"),)).fetchone() if \
         session.get("uid") else None
@app.context_processor
def ctx():
    return dict(user=me(), csrf=session.setdefault("csrf", secrets.token_hex(16)), ENT=ENT, LONG=LONG)
@app.before_request
def csrf_check():
    if request.method == "POST" and not secrets.compare_digest(request.form.get("_csrf", ""), session.get("c \
        srf", "x")):
        abort(400, "Invalid or missing form token. Go back and reload the page.")
@app.after_request
def headers(r):
    r.headers["Content-Security-Policy"] = "default-src 'self'; img-src 'self' https: data:; frame-src https \
        ://www.youtube.com https://www.youtube-nocookie.com; object-src 'none'; base-uri 'self'; frame-ances \
        tors 'self'"
    r.headers["X-Content-Type-Options"] = "nosniff"; r.headers["Referrer-Policy"] = "same-origin"
    return r

import time
from io import BytesIO
from PIL import Image
UP = os.path.join(os.path.dirname(DB), "uploads")
FAILS = {}
def save_image(file):
    """Validate by decoding, strip metadata, resize, store under a random name. Returns /media/<name> or Non \
        e."""
    try:
        raw = file.read(); im = Image.open(BytesIO(raw)); im.verify()
        im = Image.open(BytesIO(raw))
        if im.format not in ("JPEG", "PNG", "WEBP", "GIF"): return None
        im = im.convert("RGB"); im.thumbnail((1600, 1600))
    except Exception:
        return None
    os.makedirs(UP, exist_ok=True); name = secrets.token_hex(12) + ".jpg"
    im.save(os.path.join(UP, name), "JPEG", quality=85); return "/media/" + name
def drop_image(url):
    if url and url.startswith("/media/"):
        p = os.path.join(UP, os.path.basename(url))
        if os.path.exists(p): os.remove(p)
@app.route("/media/<name>")
def media(name): return send_from_directory(UP, name)
def need(role=None):
    def deco(f):
        @wraps(f)
        def w(*a, **k):
            u = me()
            if not u: return redirect(url_for("login", next=request.path))
            if role and u["role"] != role: abort(403)
            return f(*a, **k)
        return w
    return deco
def audit(action, entity):
    db().execute("INSERT INTO audit(user_id,action,entity) VALUES(?,?,?)", (session.get("uid"), action, enti \
        ty))
def safe_url(u):
    return u if u.startswith(("https://", "/")) and not u.startswith("//") else ""
# ---------- public ----------
@app.route("/")
def home():
    now = dt.datetime.utcnow().strftime("%Y-%m-%dT%H:%M")
    d = db()
    slides = d.execute("SELECT * FROM slides WHERE published=1 AND (starts_at='' OR starts_at<=?) AND (ends_ \
        at='' OR ends_at>?) ORDER BY CAST(sort_order AS INTEGER)", (now, now)).fetchall()
    today = dt.date.today().isoformat()
    hope = d.execute("SELECT * FROM hope WHERE published=1 AND publish_date<=? ORDER BY publish_date DESC LI \
        MIT 1", (today,)).fetchone()
    return render_template("home.html", slides=slides, hope=hope, live=d.execute("SELECT * FROM live_service \
        ").fetchone(),
        events=d.execute("SELECT * FROM events WHERE published=1 ORDER BY starts_at LIMIT 3").fetchall(),
        sermons=d.execute("SELECT * FROM sermons WHERE published=1 ORDER BY recorded_at DESC LIMIT 3").fetch \
            all())
@app.route("/p/<ent>")
def public_list(ent):
    if ent not in ENT: abort(404)
    order = {"events":"starts_at","sermons":"recorded_at DESC","leaders":"CAST(sort_order AS INTEGER)","hope \
        ":"publish_date DESC"}.get(ent, "id DESC")
    rows = db().execute(f"SELECT * FROM {ent} WHERE published=1 ORDER BY {order}").fetchall()
    return render_template("list.html", ent=ent, rows=rows, fields=ENT[ent][1])
@app.route("/live")
```

```
def live():
    return render_template("live.html", live=db().execute("SELECT * FROM live_service").fetchone(),
        last=db().execute("SELECT * FROM sermons WHERE published=1 ORDER BY recorded_at DESC LIMIT 1").fetch \
            one())
@app.route("/give")
def give(): return render_template("give.html", info=os.environ.get("GIVING_INFO", ""))
@app.route("/<kind>", methods=["GET", "POST"])
def request_form(kind):
    if kind not in ("prayer", "join"): abort(404)
    if request.method == "POST":
        n, c, b = (request.form.get(x, "").strip()[:2000] for x in ("name", "contact", "body"))
        if not n or not b: flash("Please fill in your name and message.")
        else:
            db().execute("INSERT INTO requests(kind,name,contact,body,consent) VALUES(?,?,?,?,?)", (kind, n, \
                 c, b, 1 if request.form.get("consent") else 0))
            db().commit(); flash("Thank you - your request was received."); return redirect(url_for("request \
                _form", kind=kind))
    return render_template("request.html", kind=kind)
# ---------- accounts ----------
@app.route("/register", methods=["GET", "POST"])
def register():
    if request.method == "POST":
        e, n, p = request.form.get("email", "").strip().lower(), request.form.get("name", "").strip(), reque \
            st.form.get("password", "")
        if "@" not in e or not n or len(p) < 10: flash("Valid email, name and a password of 10+ characters a \
            re required.")
        elif db().execute("SELECT 1 FROM users WHERE email=?", (e,)).fetchone(): flash("Could not create acc \
            ount with those details.")
        else:
            db().execute("INSERT INTO users(email,password_hash,display_name) VALUES(?,?,?)", (e, generate_p \
                assword_hash(p), n)); db().commit()
            flash("Account created. Please sign in."); return redirect(url_for("login"))
    return render_template("auth.html", mode="register")
@app.route("/login", methods=["GET", "POST"])
def login():
    if request.method == "POST":
        ip = request.remote_addr; FAILS[ip] = [t for t in FAILS.get(ip, []) if time.time() - t < 600]
        if len(FAILS[ip]) >= 5: flash("Too many attempts. Try again in 10 minutes."); return render_template \
            ("auth.html", mode="login"), 429
        u = db().execute("SELECT * FROM users WHERE email=? AND enabled=1", (request.form.get("email", "").s \
            trip().lower(),)).fetchone()
        if u and check_password_hash(u["password_hash"], request.form.get("password", "")):
            session.clear(); session["uid"] = u["id"]
            nxt = request.args.get("next", "")
            return redirect(nxt if nxt.startswith("/") and not nxt.startswith("//") else url_for("admin" if  \
                u["role"] == "ADMIN" else "member"))
        FAILS[ip].append(time.time()); flash("Incorrect email or password.")
    return render_template("auth.html", mode="login")
@app.route("/logout", methods=["POST"])
def logout(): session.clear(); return redirect("/")
@app.route("/member")
@need()
def member():
    return render_template("member.html", msgs=db().execute("SELECT * FROM messages WHERE user_id IS NULL OR \
         user_id=? ORDER BY id DESC", (session["uid"],)).fetchall())
@app.route("/sermon/<int:i>", methods=["GET", "POST"])
def sermon(i):
    s = db().execute("SELECT * FROM sermons WHERE id=? AND published=1", (i,)).fetchone() or abort(404)
    if request.method == "POST":
        if not me(): return redirect(url_for("login", next=request.path))
        r = max(1, min(5, int(request.form.get("rating", 5) or 5)))
        db().execute("INSERT INTO comments(sermon_id,user_id,text,rating) VALUES(?,?,?,?)", (i, session["uid \
            "], request.form.get("text", "")[:1000], r)); db().commit()
        return redirect(request.path)
    cm = db().execute("SELECT c.*,u.display_name FROM comments c JOIN users u ON u.id=c.user_id WHERE sermon \
        _id=? ORDER BY c.id DESC", (i,)).fetchall()
    return render_template("sermon.html", s=s, cm=cm)
@app.route("/comment/<int:i>/delete", methods=["POST"])
@need()
def comment_delete(i):
    c = db().execute("SELECT * FROM comments WHERE id=?", (i,)).fetchone() or abort(404)
    if c["user_id"] != session["uid"] and me()["role"] != "ADMIN": abort(403)
    db().execute("DELETE FROM comments WHERE id=?", (i,)); db().commit(); return redirect(f"/sermon/{c['serm \
        on_id']}")
# ---------- admin ----------
@app.route("/admin")
@need("ADMIN")
def admin():
    d = db(); counts = {k: d.execute(f"SELECT COUNT(*) FROM {k}").fetchone()[0] for k in ENT}
    return render_template("admin.html", counts=counts, new=d.execute("SELECT COUNT(*) FROM requests WHERE s \
        tatus='NEW'").fetchone()[0],
        audit=d.execute("SELECT a.*,u.email FROM audit a LEFT JOIN users u ON u.id=a.user_id ORDER BY a.id D \
            ESC LIMIT 15").fetchall())
@app.route("/admin/live", methods=["GET", "POST"])
@need("ADMIN")
def admin_live():
    if request.method == "POST":
        st = request.form.get("status"); st = st if st in ("SCHEDULED", "LIVE", "ENDED") else "SCHEDULED"
        db().execute("UPDATE live_service SET title=?,stream_url=?,status=?,scheduled_at=? WHERE id=1", (req \
            uest.form["title"][:160], safe_url(request.form.get("stream_url", "")), st, request.form.get("sc \
            heduled_at", "")))
```

```
        audit("update", "live_service"); db().commit(); flash("Live service updated.")
    return render_template("admin_live.html", live=db().execute("SELECT * FROM live_service").fetchone())
@app.route("/admin/requests", methods=["GET", "POST"])
@need("ADMIN")
def admin_requests():
    if request.method == "POST":
        db().execute("UPDATE requests SET status=? WHERE id=?", ("DONE" if request.form.get("done") else "NE \
            W", request.form["id"])); audit("update", "requests"); db().commit()
    return render_template("admin_requests.html", rows=db().execute("SELECT * FROM requests ORDER BY id DESC \
        ").fetchall())
@app.route("/admin/messages", methods=["GET", "POST"])
@need("ADMIN")
def admin_messages():
    if request.method == "POST":
        db().execute("INSERT INTO messages(user_id,subject,body) VALUES(NULL,?,?)", (request.form["subject"] \
            [:160], request.form["body"][:5000])); audit("broadcast", "messages"); db().commit(); flash("Mes \
            sage sent to all members.")
    return render_template("admin_messages.html", rows=db().execute("SELECT * FROM messages ORDER BY id DESC \
        ").fetchall())
@app.route("/admin/<ent>", methods=["GET", "POST"])
@need("ADMIN")
def admin_ent(ent):
    if ent not in ENT: abort(404)
    f, d = ENT[ent][1], db()
    if request.method == "POST":
        act, i = request.form.get("act"), request.form.get("id")
        old = d.execute(f"SELECT * FROM {ent} WHERE id=?", (i,)).fetchone() if i else None
        if act == "delete":
            if old: drop_image(old["image_url"])
            d.execute(f"DELETE FROM {ent} WHERE id=?", (i,))
        elif act == "toggle": d.execute(f"UPDATE {ent} SET published=1-published WHERE id=?", (i,))
        else:
            vals = [request.form.get(x, "").strip()[:5000] for x in f]
            for n, x in enumerate(f):
                if x in ("image_url", "action_url", "video_url"): vals[n] = safe_url(vals[n])
            k = f.index("image_url"); keep = old["image_url"] if old else ""
            up = request.files.get("image_file")
            if up and up.filename:
                new_url = save_image(up)
                if not new_url: flash("That file is not a valid JPG, PNG, WEBP or GIF image."); return redir \
                    ect(request.url)
                drop_image(keep); vals[k] = new_url
            elif request.form.get("remove_image"): drop_image(keep); vals[k] = ""
            elif keep.startswith("/media/") and not vals[k]: vals[k] = keep
            if i: d.execute(f"UPDATE {ent} SET "+",".join(f"{x}=?" for x in f)+" WHERE id=?", vals+[i])
            else: d.execute(f"INSERT INTO {ent}("+",".join(f)+") VALUES("+",".join("?"*len(f))+")", vals)  # \
                 drafts by default
        audit(act or "save", ent); d.commit(); return redirect(request.path)
    edit = d.execute(f"SELECT * FROM {ent} WHERE id=?", (request.args.get("edit"),)).fetchone() if request.a \
        rgs.get("edit") else None
    return render_template("admin_ent.html", ent=ent, rows=d.execute(f"SELECT * FROM {ent} ORDER BY id DESC" \
        ).fetchall(), fields=f, edit=edit)
init()
if __name__ == "__main__":
    app.run(host="0.0.0.0", port=int(os.environ.get("PORT", 5000)))
```

templates/admin.html

```
{% extends "base.html" %}{% block body %}{% include "admin_nav.html" %}<h1>Dashboard</h1><p>{{ new }} new pr \
    ayer/join request(s).</p><div class="grid">{% for k,n in counts.items() %}<div class="card">{{ ENT[k][0] \
     }}: <b>{{ n }}</b></div>{% endfor %}</div>
<h2>Recent admin activity</h2><ul>{% for a in audit %}<li>{{ a.created_at }} · {{ a.email }} · {{ a.action } \
    } {{ a.entity }}</li>{% endfor %}</ul>{% endblock %}
```

templates/admin_ent.html

```
{% extends "base.html" %}{% block body %}{% include "admin_nav.html" %}<h1>{{ ENT[ent][0] }}</h1>
<form method="post" class="form" enctype="multipart/form-data"><input type="hidden" name="_csrf" value="{{ c \
    srf }}">{% if edit %}<input type="hidden" name="id" value="{{ edit.id }}">{% endif %}
{% for f in fields %}<label>{{ f.replace('_',' ').title() }}{% if f in LONG %}<textarea name="{{ f }}">{{ ed \
    it[f] if edit }}</textarea>{% else %}<input name="{{ f }}" value="{{ edit[f] if edit }}"{% if f in ('sta \
    rts_at','ends_at') %} placeholder="2026-12-25T18:00"{% elif f=='publish_date' %} placeholder="2026-12-25 \
    "{% endif %}>{% endif %}</label>{% endfor %}
<label>Upload image (JPG, PNG, WEBP, GIF; max 8 MB)<input type="file" name="image_file" accept="image/*"></l \
    abel>{% if edit and edit.image_url %}<img class="thumb" src="{{ edit.image_url }}" alt=""><label class=" \
    chk"><input type="checkbox" name="remove_image"> Remove current image</label>{% endif %}
<button class="button">{{ 'Save changes' if edit else 'Save as draft' }}</button></form>
<table>{% for r in rows %}<tr><td>{% if r.image_url %}<img class="mini" src="{{ r.image_url }}" alt="">{% en \
    dif %}</td><td>{{ r['title'] or r['name'] or r['heading'] }}</td><td>{{ 'Published' if r.published else  \
    'Draft' }}</td><td class="acts"><a href="?edit={{ r.id }}">Edit</a>
{% for a,l in (('toggle','Publish/Unpublish'),('delete','Delete')) %}<form method="post"><input type="hidden \
    " name="_csrf" value="{{ csrf }}"><input type="hidden" name="id" value="{{ r.id }}"><input type="hidden" \
     name="act" value="{{ a }}"><button class="link" data-confirm="{{ 'Delete this item?' if a=='delete' }}" \
    >{{ l }}</button></form>{% endfor %}</td></tr>{% endfor %}</table>{% endblock %}
```

templates/admin_live.html

```
{% extends "base.html" %}{% block body %}{% include "admin_nav.html" %}<h1>Live service</h1><p>Set LIVE only \
     after you confirm the stream is running. Use an https YouTube embed link (youtube.com/embed/ID) to show \
     the player.</p>
<form method="post" class="form"><input type="hidden" name="_csrf" value="{{ csrf }}"><label>Title<input nam \
    e="title" value="{{ live.title }}" required></label><label>Stream URL<input name="stream_url" value="{{  \
    live.stream_url or '' }}"></label>
<label>Status<select name="status">{% for s in ('SCHEDULED','LIVE','ENDED') %}<option{{ ' selected' if live. \
    status==s }}>{{ s }}</option>{% endfor %}</select></label><label>Next service (East Africa Time)<input n \
    ame="scheduled_at" value="{{ live.scheduled_at or '' }}" placeholder="2026-10-18T09:00"></label><button  \
    class="button">Save</button></form>{% endblock %}
```

templates/admin_messages.html

```
{% extends "base.html" %}{% block body %}{% include "admin_nav.html" %}<h1>Message all members</h1><form met \
    hod="post" class="form"><input type="hidden" name="_csrf" value="{{ csrf }}"><label>Subject<input name=" \
    subject" required></label><label>Message<textarea name="body" required></textarea></label><button class= \
    "button">Send</button></form>
{% for m in rows %}<div class="card"><b>{{ m.subject }}</b> <small>{{ m.created_at }}</small></div>{% endfor \
     %}{% endblock %}
```

templates/admin_nav.html

```
<nav class="anav"><a href="/admin">Overview</a>{% for k,v in ENT.items() %}<a href="/admin/{{ k }}">{{ v[0]  \
    }}</a>{% endfor %}<a href="/admin/live">Live service</a><a href="/admin/requests">Prayer/join requests</ \
    a><a href="/admin/messages">Messages</a></nav>
```

templates/admin_requests.html

```
{% extends "base.html" %}{% block body %}{% include "admin_nav.html" %}<h1>Prayer and join requests</h1>{% f \
    or r in rows %}<div class="card"><b>{{ r.kind }}</b> · {{ r.name }} · {{ r.contact }} · {{ 'may contact' \
     if r.consent else 'no contact consent' }}<p class="pre">{{ r.body }}</p>
<form method="post"><input type="hidden" name="_csrf" value="{{ csrf }}"><input type="hidden" name="id" valu \
    e="{{ r.id }}"><label class="chk"><input type="checkbox" name="done" {{ 'checked' if r.status=='DONE' }} \
    > Handled</label><button class="link">Update</button></form></div>{% else %}<p>None yet.</p>{% endfor %} \
    {% endblock %}
```

templates/auth.html

```
{% extends "base.html" %}{% block body %}<h1>{{ 'Create account' if mode=='register' else 'Sign in' }}</h1>< \
    form method="post" class="form"><input type="hidden" name="_csrf" value="{{ csrf }}">
{% if mode=='register' %}<label>Name<input name="name" required></label>{% endif %}<label>Email<input name=" \
    email" type="email" required></label><label>Password<input name="password" type="password" required minl \
    ength="{{ 10 if mode=='register' else 1 }}"></label>
<button class="button">{{ 'Register' if mode=='register' else 'Sign in' }}</button></form><p>{% if mode=='lo \
    gin' %}<a href="/register">Create a member account</a>{% else %}<a href="/login">Sign in</a>{% endif %}< \
    /p>{% endblock %}
```

templates/base.html

```
<!doctype html><html lang="en"><head><meta charset="utf-8"><meta name="viewport" content="width=device-width \
    ,initial-scale=1">
<title>Home Church International</title><link rel="stylesheet" href="/static/css/site.css"><script src="/sta \
    tic/js/site.js" defer></script></head><body>
<header class="top"><a class="brand" href="/">HOME CHURCH <span>INTERNATIONAL</span></a>
<nav><a href="/live">Live</a><a href="/p/hope">Daily Hope</a><a href="/p/events">Events</a><a href="/p/sermo \
    ns">Sermons</a><a href="/p/leaders">Leaders</a><a href="/p/projects">Projects</a><a href="/prayer">Praye \
    r</a><a href="/give">Give</a>
{% if user %}<a href="{{ '/admin' if user.role=='ADMIN' else '/member' }}">{{ 'Admin' if user.role=='ADMIN'  \
    else 'Inbox' }}</a>
<form method="post" action="/logout"><input type="hidden" name="_csrf" value="{{ csrf }}"><button class="lin \
    k">Sign out</button></form>{% else %}<a href="/login">Sign in</a>{% endif %}</nav></header>
{% for m in get_flashed_messages() %}<p class="flash" role="status">{{ m }}</p>{% endfor %}
<main>{% block body %}{% endblock %}</main><footer>© Home Church International · A home for every nation</fo \
    oter></body></html>
```

templates/give.html

```
{% extends "base.html" %}{% block body %}<h1>Give</h1>{% if info %}<p class="pre">{{ info }}</p>{% else %}<p \
    >Giving details have not been published yet. The administrator sets them with the GIVING_INFO setting.</ \
    p>{% endif %}{% endblock %}
```

templates/home.html

```
{% extends "base.html" %}{% block body %}
<section class="hero-slider" aria-label="Church announcements" data-interval="4000">
{% for s in slides %}<article class="hero-slide{{ ' is-active' if loop.first }}">{% if s.image_url %}<img sr \
    c="{{ s.image_url }}" alt="">{% endif %}
<div class="hero-text"><h1>{{ s.title }}</h1><p>{{ s.body }}</p>{% if s.action_url %}<a class="button" href= \
    "{{ s.action_url }}">{{ s.action_label or 'Learn more' }}</a>{% endif %}</div></article>
{% else %}<article class="hero-slide is-active"><img src="/static/img/fellowship.jpg" alt=""><div class="her \
    o-text"><h1>Welcome home</h1><p>A home for every nation.</p><a class="button" href="/live">Join live ser \
    vice</a></div></article>{% endfor %}
<div class="hero-controls"><button id="prev" type="button">Previous</button><button id="toggle" type="button \
    " aria-pressed="false">Pause slides</button><button id="next" type="button">Next</button></div></section \
    >
<section class="grid"><div class="card"><h2>{% if live.status=='LIVE' %}<span class="badge">LIVE</span> {% e \
    ndif %}{{ live.title }}</h2>
<p>{% if live.status=='LIVE' %}Service is live now.{% elif live.scheduled_at %}Next service: {{ live.schedul \
    ed_at.replace('T',' ') }}{% else %}Next service to be announced.{% endif %}</p><a class="button" href="/ \
    live">Join live service</a></div>
{% if hope %}<div class="card"><h2>Daily Hope</h2><p><b>{{ hope.heading }}</b></p><blockquote>{{ hope.verse_ \
    text }} <cite>{{ hope.scripture_reference }}</cite></blockquote><p>{{ hope.prayer_prompt }}</p></div>{%  \
    endif %}</section>
<h2 class="sec">Upcoming events</h2><div class="grid">{% for e in events %}<div class="card"><h3>{{ e.title  \
    }}</h3><p>{{ e.starts_at.replace('T',' ') }} · {{ e.location }}</p><p>{{ e.description }}</p></div>{% el \
    se %}<p>No events yet.</p>{% endfor %}</div>
<h2 class="sec">Latest sermons</h2><div class="grid">{% for s in sermons %}<div class="card"><h3><a href="/s \
    ermon/{{ s.id }}">{{ s.title }}</a></h3><p>{{ s.preacher }} · {{ s.recorded_at }}</p></div>{% else %}<p> \
    No sermons yet.</p>{% endfor %}</div>
{% endblock %}
```

templates/list.html

```
{% extends "base.html" %}{% block body %}<h1>{{ ENT[ent][0] }}</h1><div class="grid">{% for r in rows %}<div \
     class="card">
{% if r['image_url'] %}<img class="thumb" src="{{ r['image_url'] }}" alt="">{% endif %}
<h3>{% if ent=='sermons' %}<a href="/sermon/{{ r.id }}">{{ r.title }}</a>{% else %}{{ r['title'] or r['name' \
    ] or r['heading'] }}{% endif %}</h3>
{% for f in fields if f not in ('title','name','heading','image_url','video_url','sort_order') and r[f] %}<p \
    >{{ r[f].replace('T',' ') }}</p>{% endfor %}</div>{% else %}<p>Nothing published yet.</p>{% endfor %}</d \
    iv>{% endblock %}
```

templates/live.html

```
{% extends "base.html" %}{% block body %}<h1>{% if live.status=='LIVE' %}<span class="badge">LIVE</span> {%  \
    endif %}{{ live.title }}</h1>
{% if live.status=='LIVE' and live.stream_url %}{% if 'youtube.com/embed/' in live.stream_url %}<iframe clas \
    s="player" src="{{ live.stream_url }}" title="Live service" allowfullscreen></iframe>{% endif %}<p><a cl \
    ass="button" href="{{ live.stream_url }}" rel="noopener">Open stream</a></p>
{% else %}<p>{{ ('Next service: ' ~ live.scheduled_at.replace('T',' ') ~ ' (East Africa Time)') if live.sche \
    duled_at else 'No service is live right now.' }}</p>
{% if last %}<p>Missed it? Watch the latest message: <a href="/sermon/{{ last.id }}">{{ last.title }}</a></p \
    >{% endif %}{% endif %}{% endblock %}
```

templates/member.html

```
{% extends "base.html" %}{% block body %}<h1>Inbox</h1>{% for m in msgs %}<div class="card"><h3>{{ m.subject \
     }}</h3><p class="pre">{{ m.body }}</p><small>{{ m.created_at }}</small></div>{% else %}<p>No messages.< \
    /p>{% endfor %}{% endblock %}
```

templates/request.html

```
{% extends "base.html" %}{% block body %}<h1>{{ 'Prayer request' if kind=='prayer' else 'Join a group' }}</h \
    1><form method="post" class="form"><input type="hidden" name="_csrf" value="{{ csrf }}">
<label>Name<input name="name" required maxlength="120"></label><label>Phone or email<input name="contact" ma \
    xlength="200"></label><label>{{ 'Request' if kind=='prayer' else 'Notes' }}<textarea name="body" require \
    d maxlength="2000"></textarea></label>
<label class="chk"><input type="checkbox" name="consent"> You may contact me about this</label><button class \
    ="button">Send</button></form>{% endblock %}
```

templates/sermon.html

```
{% extends "base.html" %}{% block body %}<h1>{{ s.title }}</h1><p>{{ s.preacher }} · {{ s.scripture }} · {{  \
    s.recorded_at }}</p>{% if s.image_url %}<img class="thumb" src="{{ s.image_url }}" alt="">{% endif %}<p> \
    {{ s.summary }}</p>
{% if s.video_url %}{% if 'youtube.com/embed/' in s.video_url %}<iframe class="player" src="{{ s.video_url } \
    }" title="{{ s.title }}" allowfullscreen></iframe>{% else %}<a class="button" href="{{ s.video_url }}" r \
    el="noopener">Watch</a>{% endif %}{% endif %}
<h2>Comments</h2>{% if user %}<form method="post" class="form"><input type="hidden" name="_csrf" value="{{ c \
```

```
    srf }}"><label>Rating<select name="rating"><option>5<option>4<option>3<option>2<option>1</select></label \
    ><label>Comment<textarea name="text" maxlength="1000"></textarea></label><button class="button">Post</bu \
    tton></form>{% else %}<p><a href="/login?next=/sermon/{{ s.id }}">Sign in</a> to comment.</p>{% endif %}
{% for c in cm %}<div class="card"><b>{{ c.display_name }}</b> · {{ c.rating }}/5<p>{{ c.text }}</p>{% if us \
    er and (user.id==c.user_id or user.role=='ADMIN') %}<form method="post" action="/comment/{{ c.id }}/dele \
    te"><input type="hidden" name="_csrf" value="{{ csrf }}"><button class="link">Delete</button></form>{% e \
    ndif %}</div>{% endfor %}{% endblock %}
```

static/css/site.css

```
:root{--navy:#14284b;--gold:#c9973a;--ink:#202938;--paper:#f5f7fb;--line:#dce2eb}
*{box-sizing:border-box}body{margin:0;color:var(--ink);background:var(--paper);font:16px/1.5 system-ui,sans- \
    serif}main{max-width:1100px;margin:auto;padding:1rem}
a{color:var(--navy)}.top{display:flex;flex-wrap:wrap;gap:1rem;justify-content:space-between;align-items:cent \
    er;padding:1rem 2rem;background:var(--navy)}.top a,.top .link{color:#fff;text-decoration:none;margin-rig \
    ht:.8rem;font-weight:600}
.brand{font-weight:800;letter-spacing:.04em;font-size:1.1rem}.brand span{color:var(--gold)}nav{display:flex; \
    flex-wrap:wrap;align-items:center}nav form{display:inline}
.button,button{display:inline-block;border:0;border-radius:10px;padding:.7rem 1.1rem;background:var(--gold); \
    color:#171717;font:inherit;font-weight:700;cursor:pointer;text-decoration:none}
button.link{background:none;padding:0;color:var(--navy);text-decoration:underline;font-weight:600}.top butto \
    n.link{color:#fff}
:focus-visible{outline:3px solid #2366d1;outline-offset:3px}
.hero-slider{position:relative;min-height:440px;border-radius:20px;overflow:hidden;background:var(--navy);is \
    olation:isolate}
.hero-slide{position:absolute;inset:0;display:flex;align-items:center;opacity:0;transition:opacity .9s ease; \
    pointer-events:none}.hero-slide.is-active{opacity:1;pointer-events:auto}
.hero-slide img{position:absolute;inset:0;width:100%;height:100%;object-fit:cover;z-index:-2}.hero-slide::be \
    fore{content:"";position:absolute;inset:0;background:linear-gradient(90deg,#0a1428e0,#0a142859);z-index: \
    -1}
.hero-text{padding:clamp(24px,6vw,72px);color:#fff;max-width:640px}.hero-text h1{font-size:clamp(2rem,6vw,3. \
    4rem);margin:0 0 .5rem}
.hero-controls{position:absolute;bottom:12px;right:12px;display:flex;gap:.5rem}.hero-controls button{backgro \
    und:#ffffffd9;padding:.4rem .8rem}
.grid{display:grid;grid-template-columns:repeat(auto-fit,minmax(260px,1fr));gap:1rem;margin:1rem 0}.card{bac \
    kground:#fff;border:1px solid var(--line);border-radius:16px;padding:1rem;box-shadow:0 8px 24px #14284b0 \
    a}
.thumb{width:100%;border-radius:10px}.badge{background:#c62828;color:#fff;padding:.1rem .6rem;border-radius: \
    6px;font-size:.8em}.sec{margin-top:2rem}.player{width:100%;aspect-ratio:16/9;border:0;border-radius:12px \
    }
.form{max-width:560px}label{display:grid;gap:.3rem;margin:.8rem 0;font-weight:650}.chk{display:flex;gap:.5re \
    m;align-items:center}input,textarea,select{width:100%;padding:.7rem;border:1px solid #aab5c6;border-radi \
    us:8px;font:inherit}.chk input{width:auto}textarea{min-height:100px}
.flash{background:#fff7e0;border:1px solid var(--gold);padding:.7rem 1rem;margin:0;text-align:center}.anav{d \
    isplay:flex;flex-wrap:wrap;gap:.8rem;padding:.6rem 0;border-bottom:1px solid var(--line)}
table{width:100%;border-collapse:collapse;background:#fff;margin-top:1rem}td{padding:.6rem;border-bottom:1px \
     solid var(--line)}.acts{display:flex;gap:.8rem;flex-wrap:wrap}.acts form{margin:0}.pre{white-space:pre- \
    wrap}footer{text-align:center;padding:2rem;color:#596579}
@media(prefers-reduced-motion:reduce){.hero-slide{transition:none}}
.mini{width:56px;height:40px;object-fit:cover;border-radius:6px}
```

static/js/site.js

```
(function(){
var box=document.querySelector(".hero-slider");
if(box){var s=[].slice.call(box.querySelectorAll(".hero-slide")),i=0,t=null,
paused=matchMedia("(prefers-reduced-motion: reduce)").matches,tg=document.getElementById("toggle");
function show(n){s[i].classList.remove("is-active");i=(n+s.length)%s.length;s[i].classList.add("is-active")}
function run(){clearInterval(t);if(!paused&&s.length>1)t=setInterval(function(){show(i+1)},+box.dataset.inte \
    rval||4000)}
document.getElementById("next").onclick=function(){show(i+1)};document.getElementById("prev").onclick=functi \
    on(){show(i-1)};
tg.textContent=paused?"Resume slides":"Pause slides";tg.setAttribute("aria-pressed",String(paused));
tg.onclick=function(){paused=!paused;tg.textContent=paused?"Resume slides":"Pause slides";tg.setAttribute("a \
    ria-pressed",String(paused));run()};run()}
document.addEventListener("click",function(e){var m=e.target.getAttribute&&e.target.getAttribute("data-confi \
    rm");if(m&&!confirm(m))e.preventDefault()});
})();
```

tests/test_app.py

```
import os, re, io, tempfile, importlib, pytest
from PIL import Image
D = tempfile.mkdtemp(); os.environ.update(DB_PATH=D + "/t.db", ADMIN_EMAIL="a@x.org", ADMIN_PASSWORD="longpa \
    ssword123")
app_mod = importlib.import_module("app"); app = app_mod.app
tok = lambda r: re.search(r'name="_csrf" value="(\w+)"', r.get_data(as_text=True)).group(1)
def png(): b = io.BytesIO(); Image.new("RGB", (40, 30), "navy").save(b, "PNG"); b.seek(0); return b
def login(c, e="a@x.org", p="longpassword123", ip="1.1.1.1"):
    t = tok(c.get("/login")); return c.post("/login", data=dict(_csrf=t, email=e, password=p), environ_overr \
        ides={"REMOTE_ADDR": ip})
@pytest.fixture
def admin():
    c = app.test_client(); assert login(c).status_code == 302; return c
```

```
def test_public_pages():
    c = app.test_client()
    for p in ["/", "/live", "/give", "/prayer", "/join", "/register", "/login"] + [f"/p/{k}" for k in app_mo \
        d.ENT]: assert c.get(p).status_code == 200, p
def test_admin_protected_and_csrf(admin):
    assert app.test_client().get("/admin").status_code == 302
    assert app.test_client().get("/admin/slides").status_code == 302
    assert admin.post("/admin/slides", data=dict(title="x")).status_code == 400
def test_rate_limit():
    c = app.test_client()
    for _ in range(5): assert login(c, p="wrong", ip="9.9.9.9").status_code == 200
    assert login(c, ip="9.9.9.9").status_code == 429
@pytest.mark.parametrize("ent", list(app_mod.ENT))
def test_upload_replace_remove_delete_each_section(admin, ent):
    f = app_mod.ENT[ent][1]; title = "name" if "name" in f else "heading" if "heading" in f else "title"
    page = f"/admin/{ent}"; t = tok(admin.get(page))
    r = admin.post(page, data=dict(_csrf=t, **{title: "T1"}, image_file=(png(), "a.png")), content_type="mul \
        tipart/form-data"); assert r.status_code == 302
    row = app_mod.sqlite3.connect(app_mod.DB).execute(f"SELECT id,image_url FROM {ent} WHERE {title}='T1'"). \
        fetchone()
    assert row[1].startswith("/media/") and os.path.exists(app_mod.UP + "/" + row[1][7:]); assert admin.get( \
        row[1]).status_code == 200
    t = tok(admin.get(page)); admin.post(page, data=dict(_csrf=t, id=row[0], **{title: "T1"}, remove_image=" \
        on"))
    assert not os.path.exists(app_mod.UP + "/" + row[1][7:])
    t = tok(admin.get(page)); admin.post(page, data=dict(_csrf=t, id=row[0], **{title: "T1"}, image_file=(pn \
        g(), "b.png")), content_type="multipart/form-data")
    url = app_mod.sqlite3.connect(app_mod.DB).execute(f"SELECT image_url FROM {ent} WHERE id=?", (row[0],)). \
        fetchone()[0]; assert url
    t = tok(admin.get(page)); admin.post(page, data=dict(_csrf=t, id=row[0], act="delete"))
    assert not os.path.exists(app_mod.UP + "/" + url[7:])
def test_fake_image_rejected(admin):
    t = tok(admin.get("/admin/slides"))
    r = admin.post("/admin/slides", data=dict(_csrf=t, title="Bad", image_file=(io.BytesIO(b"<script>alert(1 \
        )</script>"), "x.png")), content_type="multipart/form-data", follow_redirects=True)
    assert "not a valid" in r.get_data(as_text=True)
def test_slide_publish_xss_and_url(admin):
    t = tok(admin.get("/admin/slides")); admin.post("/admin/slides", data=dict(_csrf=t, title="<b>Hello</b>" \
        , action_url="javascript:alert(1)", action_label="Go"))
    t = tok(admin.get("/admin/slides")); i = app_mod.sqlite3.connect(app_mod.DB).execute("SELECT id FROM sli \
        des WHERE title LIKE '%Hello%'").fetchone()[0]
    admin.post("/admin/slides", data=dict(_csrf=t, id=i, act="toggle")); h = admin.get("/").get_data(as_text \
        =True)
    assert "&lt;b&gt;Hello" in h and "javascript:" not in h
def test_live_member_comment_flow(admin):
    t = tok(admin.get("/admin/live")); admin.post("/admin/live", data=dict(_csrf=t, title="Svc", status="LIV \
        E", stream_url="https://www.youtube.com/embed/abc"))
    assert "LIVE" in admin.get("/live").get_data(as_text=True)
    t = tok(admin.get("/admin/sermons")); admin.post("/admin/sermons", data=dict(_csrf=t, title="S1"))
    sid = app_mod.sqlite3.connect(app_mod.DB).execute("SELECT id FROM sermons").fetchone()[0]
    t = tok(admin.get("/admin/sermons")); admin.post("/admin/sermons", data=dict(_csrf=t, id=sid, act="toggl \
        e"))
    m = app.test_client(); t = tok(m.get("/register")); m.post("/register", data=dict(_csrf=t, name="Jo", em \
        ail="jo@x.org", password="longpassword1"))
    assert login(m, "jo@x.org", "longpassword1", "2.2.2.2").status_code == 302
    assert m.get("/admin").status_code == 403
    t = tok(m.get(f"/sermon/{sid}")); m.post(f"/sermon/{sid}", data=dict(_csrf=t, text="Amen", rating="5"))
    page = m.get(f"/sermon/{sid}"); assert "Amen" in page.get_data(as_text=True)
    cid = app_mod.sqlite3.connect(app_mod.DB).execute("SELECT id FROM comments").fetchone()[0]
    assert m.post(f"/comment/{cid}/delete", data=dict(_csrf=tok(page))).status_code == 302
```